

Residual Cascade: Boosting-Style Staged MLP Forecaster (PyTorch)

Model Research Report (Sample spec, PyTorch) — `model_type`: TimeSeries — a differentiable analogue of gradient boosting: a linear mean head plus a cascade of small MLP stages that each fit the residual

1. Objective and model card

Gradient-boosted trees win many tabular competitions by adding many weak learners, each fitting the residual of the ensemble so far. This report specifies a fully differentiable torch analogue: a linear 'mean head' produces the initial prediction, then `n_stages` small MLP stages each add a gated correction computed from the flattened input window. It is a single forward pass (no sequential fitting loop), so it trains end-to-end with one optimizer while keeping the boosting intuition. Model card:

```
model_name: ResidualCascadeModel
model_type: TimeSeries
description: Boosting-style residual cascade. Flattens the input window, applies a linear mean head, then adds n_stages gated two-layer Tanh MLP corrections, each scaled by a shared learned gate eta; returns a scalar forecast as (batch_size, 1).
hyperparameters: n_stages = 5; hidden = 8
training_hyperparameters: n_epochs = 40; lr = 1e-3; early_stop = 10; batch_size = 128; weight_decay = 0
```

2. Integration contract (mandatory)

The deliverable is ONE Python file named `model.py` containing exactly one class subclassing `torch.nn.Module` and the module-level assignment `model_cls = <YourClass>`. The automated runner applies this contract verbatim:

```
import torch
from model import model_cls
m = model_cls(num_features=10, num_timesteps=4)
for _, param in m.named_parameters():
    param.data.fill_(1.0)
out = m(torch.full((32, 4, 10), 1.0))
assert out.shape == (32, 1) and torch.isfinite(out).all()
```

Consequences: (a) `__init__(self, num_features, num_timesteps, **kwargs)` with defaults for everything else; (b) forward receives shape `(batch_size, num_timesteps, num_features)` and returns `(batch_size, 1)` - do NOT permute the input, timesteps are already axis 1; (c) with every `nn.Parameter` overwritten by 1.0 and a constant all-ones input the output must stay finite: no division by parameters, no BatchNorm (zero-variance batch gives NaN), prefer LayerNorm with eps or tanh/sigmoid gates; (d) fixed constants belong in `register_buffer` (buffers are NOT overwritten); (e) no side effects at import time, no try-except, no top-level main; the demo of this report must live inside an `if __name__ == '__main__':` guard. Only torch and the Python standard library may be imported. `model_type`: TimeSeries.

3. Model formulation

`forward(x)` with `x` of shape `(B, T, F)`. Flatten `h = x.reshape(B, T*F)` (an `nn.Flatten` is fine). Let `in_dim = T*F` (compute it from `num_features * num_timesteps` in `__init__`). Base prediction `f = mean_head(h)` with `mean_head` an `nn.Linear(in_dim, 1)`. Gate `eta = sigmoid(p_eta)` for a scalar `nn.Parameter p_eta`. The cascade is an `nn.ModuleList` of `n_stages` stages, each stage = `nn.Sequential(nn.Linear(in_dim, hidden), nn.Tanh(), nn.Linear(hidden, 1))`. Accumulate `f = f + eta * stage(h)` over all stages and return `f` as `(B, 1)`. Every stage sees the SAME flattened input `h` (this is the boosting 'weak learner on the features' analogue); the residual-fitting behaviour emerges from training.

4. Stability requirements

With every parameter overwritten by 1.0 and an all-ones input: `mean_head(h)` is a finite scalar, each stage output is bounded by the Tanh in `[-1, 1]` after its final linear layer is itself bounded for the fixed all-ones `h`, and `eta = sigmoid(1) ~ 0.731`, so the sum of 5 gated corrections stays finite. No BatchNorm and no division anywhere. `in_dim` must be derived from the constructor arguments so `model_cls(num_features=10, num_timesteps=4)` yields `in_dim = 40` automatically.

5. Demo data and evaluation (`__main__` guard only)

Inside the guard build a synthetic regression with `torch.manual_seed(7)`: $N = 5000$ windows, `num_timesteps = 4`, `num_features = 10`, entries $\sim N(0, 1)$. Label $y = 0.6 * \text{sum of the first 5 flattened features} + \sin(f_5 * f_6) + 0.3 * \tanh(f_7) + N(0, 0.1)$ noise, i.e. a linear part plus interaction and saturation nonlinearities. Chronological 70/15/15 split. Train the cascade (`num_timesteps = 4`, `num_features = 10`) with Adam (lr $1e-3$, batch 128, max 40 epochs) on MSE. Compare test MSE against a single `nn.Linear(T * F, 1)` trained the same way. Reference run: cascade ~ 1.07 vs linear ~ 2.66 , with $\text{Var}(y_{\text{test}}) \sim 2.29$. Success: the cascade test MSE is strictly lower than the linear baseline's.

6. Reference

J.H. Friedman. Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics* 29(5), 2001. T. Chen, C. Guestrin. XGBoost: A Scalable Tree Boosting System. KDD 2016.